

Two-Tier File Upload & Display

Amazon EC2 • Amazon S3 • IAM Roles • Launch Templates

Project Overview

This lab deploys two independent Node.js/Express web applications on separate Amazon EC2 instances. The Uploader writes a text file to an Amazon S3 bucket; the Viewer retrieves and displays the same object. Each instance uses a dedicated IAM role with only the permissions it requires. The application source files are embedded in EC2 launch-template user-data rather than fetched from Git or S3 at boot.

Implementation Summary

AWS Region	us-east-2
Instances	Uploader EC2 / Viewer EC2 (t3.micro)
Application	Node.js / Express on port 3000
Storage object	shared.txt in S3
Uploader permission	s3:PutObject on shared.txt
Viewer permissions	s3:GetObject on shared.txt; s3:ListBucket on bucket
Versioning	Uploader launch template Versions 1 and 2
Cleanup	EC2 instances terminated; bucket, roles and launch templates removed

Source Code

Repository: https://github.com/Himanshuchaudhary17/Hchaudhary1/tree/main/Step05_EC2_S3_TwoTier

The repository includes both app.js/package.json pairs, create_app_stack.sh, delete_app_stack.sh, and IAM policy files. Local .env settings and SSH private keys were excluded from the Git commit.

1. Application Files and Deployment

Figure 1. Local Uploader and Viewer application files

```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier/lab03/viewer-app# cd /Hchaudhary1/Step05_EC2_S3_TwoTier
ls -l lab03/uploader-app/
ls -l lab03/viewer-app/
total 8
-rw-r--r-- 1 root root 2064 Oct  8 02:17 app.js
-rw-r--r-- 1 root root 189 Oct  8 02:16 package.json
total 8
-rw-r--r-- 1 root root 980 Oct  8 02:20 app.js
-rw-r--r-- 1 root root 157 Oct  8 02:19 package.json
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier#
```

Both application folders contain app.js and package.json.

Figure 2. EC2 deployment and public IP addresses

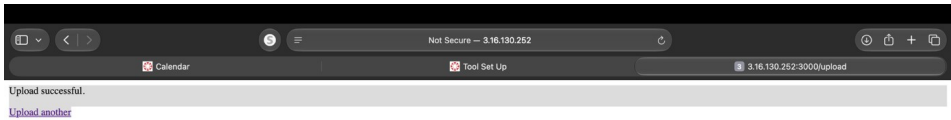
```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# cd /Hchaudhary1/Step05_EC2_S3_TwoTier

./create_app_stack.sh
== Checking S3 bucket ==
{
  "BucketArn": "arn:aws:s3:::itmo-444-544-hchaudhary1-fileshare",
  "BucketRegion": "us-east-2",
  "AccessPointAlias": false
}
Bucket already exists.
== Configuring IAM roles ==
Waiting for IAM propagation...
== Building EC2 user-data ==
== Creating EC2 launch templates ==
Created itmo-444-544-uploader-lt
Created itmo-444-544-viewer-lt
== Launching EC2 instances ==
Launched uploader: i-01322fb2d73c44d9d
Launched viewer: i-0c90550fefc11dec8
== Waiting for EC2 instances ==
== EC2 Instance Details ==
-----
|                               DescribeInstances                               |
|-----+-----+-----+-----+|
| uploader-app | i-01322fb2d73c44d9d | 3.16.130.252 | running |
| viewer-app   | i-0c90550fefc11dec8 | 3.147.242.99 | running |
|-----+-----+-----+-----+|
Wait 60-90 seconds for applications to install.
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier#
```

The create script reports an uploader and viewer instance in the running state.

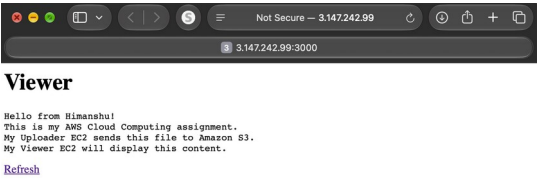
2. Successful Upload and Display

Figure 3. Text file uploaded successfully



The Uploader confirms that the file was saved successfully.

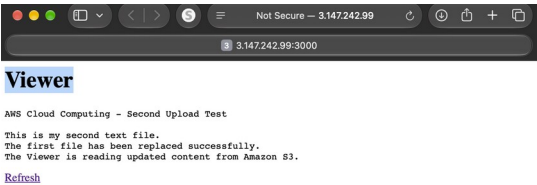
Figure 4. Viewer displays first uploaded file



The Viewer reads shared.txt from S3 and displays the initial contents.

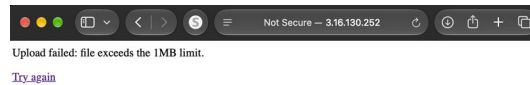
3. Live Updates and Input Validation

Figure 5. Viewer displays updated second upload



After a second text file was uploaded, refreshing the Viewer displayed the new content.

Figure 6. Oversized file rejected



A text file exceeding the 1 MB limit produces the expected rejection message.

4. File-Type Validation and Templates

Figure 7. Non-text file rejected with HTTP 415

```
(base) himanshuprince@Himanshus-MacBook-Pro ~ % curl -i -F "txtfile=@$HOME/Desktop/large-test.txt;filename=test.pdf;type=application/pdf" http://3.16.130.252:3000/upload
HTTP/1.1 100 Continue

HTTP/1.1 415 Unsupported Media Type
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 75
ETag: W/"4b-CxhTw5pNsCMAKP78QbFNYNdmh1E"
Date: Thu, 08 Oct 2026 03:01:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5

(base) himanshuprince@Himanshus-MacBook-Pro ~ %
```

A curl multipart upload labeled application/pdf was rejected by the server as an unsupported media type.

Figure 8. Both EC2 launch templates

```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# aws ec2 describe-launch-templates \
  --launch-template-names "$UPLOADER_LT_NAME" "$VIEWER_LT_NAME" \
  --query "LaunchTemplates[*].[LaunchTemplateName,LatestVersionNumber]" \
  --output table \
  --region us-east-2
-----
| DescribeLaunchTemplates |
+-----+-----+
| itmo-444-544-uploader-lt | 1 |
| itmo-444-544-viewer-lt  | 1 |
+-----+-----+
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier#
```

Uploader and Viewer launch templates appear in AWS.

5. Launch Template Versioning

Figure 9. Uploader launch template Versions 1 and 2

```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# aws ec2 describe-launch-template-versions \
  --launch-template-name "$UPLOADER_LT_NAME" \
  --query "LaunchTemplateVersions[*].[VersionNumber,CreateTime]" \
  --output table \
  --region "$AWS_REGION"
-----
| DescribeLaunchTemplateVersions |
+-----+-----+
| 2 | 2026-10-08T03:05:44+00:00 |
| 1 | 2026-10-08T02:43:57+00:00 |
+-----+-----+
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier#
```

The AWS CLI shows two published versions of the Uploader launch template.

Figure 10. Updated Uploader runs Version 2



A newly launched EC2 instance shows the changed Uploader - Version 2 heading.

6. Least-Privilege IAM Permissions

Figure 11. Uploader IAM role permissions

```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# cat uploader_instance_id.txt
cat viewer_instance_id.txt
i-0b5727e4294466b13
i-0c90550fefc11dec8
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# aws iam get-role-policy \
--role-name itmo-444-544-uploader-role \
--policy-name uploader-put-only
{
  "RoleName": "itmo-444-544-uploader-role",
  "PolicyName": "uploader-put-only",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "s3:PutObject"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-hchaudhary1-fileshare/shared.txt"
      }
    ]
  }
}
```

Uploader role grants PutObject for the exact S3 key shared.txt.

Figure 12. Viewer IAM role permissions

```
root@5dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# aws iam get-role-policy \
--role-name itmo-444-544-viewer-role \
--policy-name viewer-read-only
{
  "RoleName": "itmo-444-544-viewer-role",
  "PolicyName": "viewer-read-only",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "s3:GetObject"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-hchaudhary1-fileshare/shared.txt"
      },
      {
        "Effect": "Allow",
        "Action": [
          "s3:ListBucket"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-hchaudhary1-fileshare"
      }
    ]
  }
}
```

Viewer role grants GetObject for shared.txt and ListBucket for the S3 bucket.

7. Cleanup and GitHub Submission

The cleanup script terminated both remaining EC2 instances, deleted both launch templates, removed the IAM configuration, and deleted the shared.txt object. Follow-up AWS CLI checks confirmed both instances terminated, the S3 bucket was no longer listed, and both IAM roles returned NoSuchEntity.

Resource	Status
Remaining Uploader	Terminated (i-0b5727e4294466b13)
Viewer	Terminated (i-0c90550fefd11dec8)
S3 bucket	Not listed in aws s3 ls
Uploader IAM role	NoSuchEntity
Viewer IAM role	NoSuchEntity
Launch templates	Deletion confirmed by delete-launch-template output

Figure 13. AWS cleanup script successfully completed

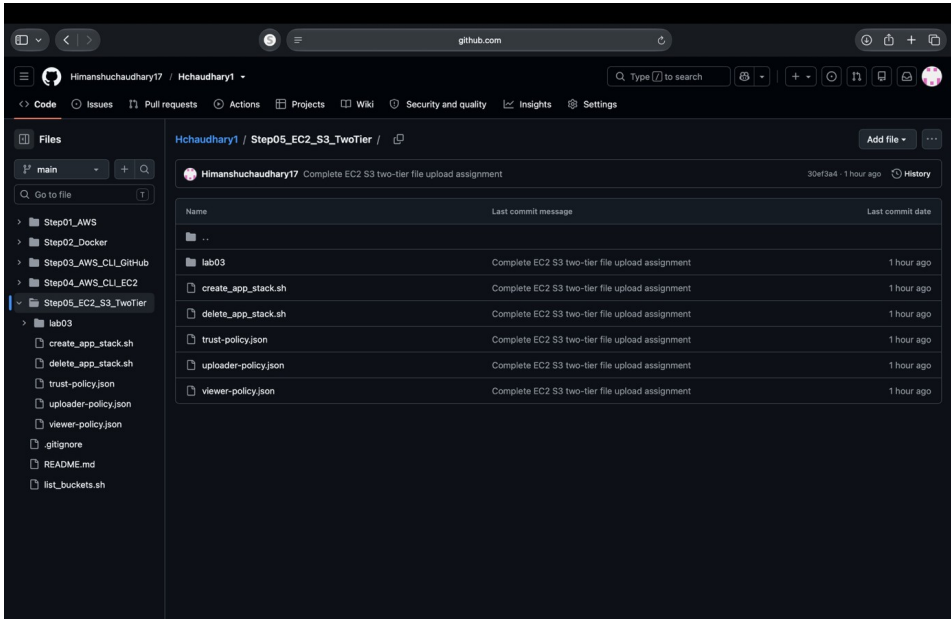
```

$3 delete_app_stack.sh
FINAL SYNTAX: OK
root@6dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier# cd /Hchaudhary1/Step05_EC2_S3_TwoTier
~/delete_app_stack.sh
== Terminating EC2 instances ==
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0b5727e4294466b13",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    },
    {
      "InstanceId": "i-0c90550fefd11dec8",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
Both EC2 instances terminated.
== Deleting launch templates ==
{
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-08e7e4cbc53228f65",
    "LaunchTemplateName": "itmo-444-544-uploader-lt",
    "CreateTime": "2020-10-08T02:43:57+00:00",
    "CreatedBy": "arn:aws:iam::174309189225:user/labuser",
    "DefaultVersionNumber": 1,
    "LatestVersionNumber": 2,
    "Operator": {
      "Managed": false
    }
  }
}
{
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-0ecc28636823e1abe",
    "LaunchTemplateName": "itmo-444-544-viewer-lt",
    "CreateTime": "2020-10-08T02:43:58+00:00",
    "CreatedBy": "arn:aws:iam::174309189225:user/labuser",
    "DefaultVersionNumber": 1,
    "LatestVersionNumber": 1,
    "Operator": {
      "Managed": false
    }
  }
}
Launch template cleanup finished.
== Removing IAM profiles and roles ==
IAM cleanup finished.
== Emptying and deleting S3 bucket ==
delete: s3://itmo-444-544-hchaudhary1-filesshare/shar
Cleanup complete.
root@6dc0b0c9efc3:/Hchaudhary1/Step05_EC2_S3_TwoTier#

```

The cleanup script reports termination and resource cleanup.

Figure 14. GitHub repository with Step05 files



The Step05_EC2_S3_TwoTier folder and required source files appear on GitHub.

8. Required Written Explanations

Why the Uploader and Viewer Use Separate IAM Roles

The Uploader and Viewer use separate IAM roles because they perform different tasks and only need access to specific AWS resources. The Uploader's job is to send the shared.txt file to Amazon S3, so it only has s3:PutObject permission. On the other hand, the Viewer needs to read and display the uploaded file, so it has s3:GetObject and s3:ListBucket permissions. By giving each application only the permissions it needs, we improve security and prevent unnecessary access. This also helps protect the system if either EC2 instance is compromised.

Why Embedding Code in User-Data Removes Code-Fetch Credentials

Instead of downloading the application code from GitHub or S3 every time an EC2 instance starts, I included the application files directly in the EC2 launch template's user-data. When the instance launches, the startup script automatically creates the required files and sets up the application. This means the instance does not need GitHub login credentials or additional S3 permissions to download the code. It makes the deployment process simpler and more secure because the IAM roles are only used for uploading and reading shared.txt from Amazon S3.

Submission Reference

GitHub: https://github.com/Himanshuchaudhary17/Hchaudhary1/tree/main/Step05_EC2_S3_TwoTier